The RLVR Arena

Train a language model with GRPO to solve Countdown number puzzles. Submit your checkpoint. It gets re-run on a secret test set. The result goes on a leaderboard.

The throughline of this entire series closes here. Every lecture traced one recursion: $V(s) = r + \gamma V(s')$. We solved it with dynamic programming (L1), approximated it with a neural network (L2), climbed it directly with policy gradients (L3), stabilised the climb with PPO's clip (L4), and applied it to a language model with RLHF (L5). In L6 we stripped away the critic entirely and replaced it with a group of sampled answers — that was GRPO. Now you implement it for real, on a real model, on a real verifiable task, and compete on a leaderboard.

The Task – Countdown

You are given a list of numbers and a target. Combine **all** the numbers — each used exactly once — with $+ \; - \; \times \; \div$ and parentheses to reach the target.

Example. Numbers: **3, 7, 8, 2** Target: **24**

$$\underbrace{(8-2)}_{6} \times \underbrace{(7-3)}_{4} = 24 \quad \checkmark \quad \text{uses 8, 2, 7, 3 exactly once.}$$

Your model must reason inside `<think>…</think>` and give its final expression inside `<answer>…</answer>`. That is the entire output contract.

Why this task? Correctness is a five-line arithmetic checker — no LLM judge, no rubric, no human in the loop. The task is combinatorial so there is nothing to memorise from the training set. And a 0.5B model that has never seen this task goes from near-zero to genuine reasoning under GRPO — that is the "aha moment" you are here to produce.

What You Implement

Open `train_grpo.py`. Two functions are marked **[TODO]**: everything else is **[PROVIDED]**.

TODO 1 – Group-Relative Advantage [12]

Input. A flat tensor of rewards of shape $[num\_puzzles \times group]$, where the k completions for each puzzle are grouped consecutively.
Output. An advantage tensor of the same shape.
The single idea of L6: within each group, how much better (or worse) was this answer compared to the *rest of the group*? That relative score — not the absolute reward — is the advantage. No critic. No baseline model. Just the group.
Review the L6 slides before writing this function.

TODO 2 – GRPO Loss [13]

Inputs. Per-token log-probs under the policy ($\log \pi_\theta$), the behaviour policy ($\log \pi_{\text{old}}$), and the frozen reference ($\log \pi_{\text{ref}}$); a completion mask; and the sequence-level advantages from TODO 1.
Output. A scalar loss.
Two ideas from lecture combine here:

- **L4 (PPO):** the clipped importance-sampling surrogate. Compute per-token probability ratios and clip them so no single step moves too far.
- **L5 (RLHF):** a KL penalty to the frozen reference keeps the model from drifting away from its starting point.

Average the per-token objective over actual completion tokens (use the mask). Return the *negative* (you minimise; gradient ascent happens automatically). The KL estimator to use is the k3 form from the GRPO paper: $\exp(\log \pi_{\text{ref}} - \log \pi_\theta) - (\log \pi_{\text{ref}} - \log \pi_\theta) - 1$.

That is all. The rollout (sampling a group of completions and computing rewards), the per-token log-prob helper, the training loop, checkpointing, and evaluation harness are fully provided.

Files You Are Given

File	Purpose
`countdown.py`	The task engine. Contains the exact verifier used to score your submission. Read it — especially `verify()` and `reward()`.
`train_grpo.py`	GRPO training starter. Fill the two [TODO] functions. Everything else runs.
`run_model.py`	Turn a saved checkpoint into `predictions.jsonl` for self-scoring.
`evaluate.py`	Score a predictions file → accuracy breakdown. Run this on `dev_public` to see your number before submitting.
`data/train.jsonl`	4 000 puzzles to train on (easy / medium / hard mix).
`data/dev_public.jsonl`	300 puzzles to self-score. The leaderboard uses the *same metric* on a larger held-out set.
`requirements.txt`	Python dependencies.

Quick start.

```
pip install -r requirements.txt

# Smoke-test (CPU, tiny model, 2 steps -- proves your code runs):
python train_grpo.py --model sshleifer/tiny-gpt2 \
    --steps 2 --group 4 --bsz 2 --smoke

# Self-score on the public dev set:
python run_model.py --model ./model \
    --test data/dev_public.jsonl --out dev_preds.jsonl
python evaluate.py --test data/dev_public.jsonl \
    --predictions dev_preds.jsonl --name yourname
```

The Base Model and the Rules

Rule	Detail
Base model	`Qwen/Qwen2.5-0.5B-Instruct` (frozen starting point, no exceptions)
Method	GRPO or any policy-gradient RL you implement. **No SFT on solution traces** — the dataset contains only puzzles, no worked answers.
No tools	The model reasons in tokens only. No calculator, no code execution, no API calls.
Eval config	Greedy decode, `max_new_tokens=640`, the canonical prompt in `countdown.py` — identical for everyone. Deterministic $\Rightarrow$ reproducible $\Rightarrow$ fair.
Open track	Optional: any base model $\leq$ 3B, ranked on a separate board. Your reward curve must show GRPO is doing the work.

Fixed base model = fixed starting line. The leaderboard measures your *RL*, not who has the bigger GPU or a stronger base. Everyone starts from exactly the same 0.5B checkpoint.

The Leaderboard [LEADERBOARD]

You submit a checkpoint. We load it, run `run_model.py` on a **private test set you have never seen**, and `evaluate.py` produces the official score. Your self-reported accuracy is never trusted — only the re-run counts.

Rank key	Metric	Notes
Primary [LEADERBOARD]	Accuracy on full private test	The headline number
Tie-break 1	Accuracy on *hard* split (5 numbers)	Rewards genuine reasoning
Tie-break 2	Avg tokens when correct (fewer wins)	Efficiency, no rambling
Diagnostic	Format compliance rate	Did it learn the protocol?

Reference points seeded on the board: (1) the untrained base model (the floor — beat this to pass), and (2) a reference GRPO run (the target — beat this to compete).

The cold-start warning. GRPO typically shows *near-zero accuracy for the first 400–600 steps* — this is normal. The model is learning the output format before it can reason. The "aha moment" (reward suddenly climbing) arrives between step 600 and 1 000 for most correct implementations. Do not panic if your curve is flat early. Run longer if you can.

What to Submit

1. **`model/`** — your fine-tuned checkpoint. Must load with `AutoModelForCausalLM.from_pretrained("./model")`. Full weights *or* a LoRA adapter with the base model ID.
2. **`dev_preds.jsonl`** — your predictions on `dev_public.jsonl`, generated by `run_model.py`. Used for a reproducibility check: re-running your checkpoint must give the same accuracy within $\pm 0.3\%$.
3. **`train_grpo.py`** — your filled-in training code. The TODOs must be implemented; the rest may be modified.
4. **`reward_curve.png`** — a plot of mean reward vs. training step. Export it from your training log.
5. **Report (≤ 1 page)** — what you tried, the reward curve interpreted, one ablation (e.g. group size, KL β, learning rate), one failure mode you encountered.

Grading

Component	Points
Working checkpoint: loads, runs, and beats the untrained floor	40
GRPO implemented correctly (both TODOs, read in your code)	25
Report: reward curve + one ablation + one failure mode	20
Reproducibility: `dev_preds` match our re-run of your checkpoint	10
Leaderboard placement (top 3 gets full 5; everyone who submits gets 3)	5
Total	**100**

Compute

Colab T4 (free). The 0.5B model fits on a 16 GB T4 at `float16`. Use `--group 4 --bsz 4` (16 sequences per step). Call `model.gradient_checkpointing_enable()` after loading to halve activation memory. Checkpoint to Drive to survive disconnects. One full run at 1 000 steps takes roughly 2–3 hours.

Modal (recommended). You have access to the `teamvizuara` Modal profile — token and instructions are on the course portal. Use `A10G` (`gpu="A10G"`) with `--group 8 --bsz 2`. A 1 000-step run finishes in about 2 hours.

Timeline

Day	Milestone
1	Run the smoke-test (CPU, tiny-gpt2, 2 steps). Read `countdown.py` end to end. Run the *untrained* base model through `evaluate.py` on dev — see the floor.
2–3	Implement both TODOs. Run on easy puzzles only (`data/train.jsonl` filtered). Get the reward moving.
4–5	Full difficulty mix. Tune: group size, KL β, learning rate. Log everything.
6	Lock a checkpoint. Generate `dev_preds.jsonl`. Write the report.
7	Submit. Leaderboard goes live.

One last thing

The verifier in `countdown.py` uses *exact rational arithmetic* (`fractions.Fraction`) and a whitelisted-AST evaluator — it never calls `eval()` on your model's output. A model that tries to write `__import__('os').system('...')` inside its `<answer>` tag gets scored *invalid*, not executed. This is a good design principle to carry forward.

Good luck. The throughline was $V(s) = r + \gamma V(s')$. Now make it real.